

Sri Eshwar College of Engineering
Kinathukadavu,Coimbatore-641202
(Approved By AICTE,New Delhi &Affiliated to Anna University,Chennai)



U23CS382-PYTHON PROGRAMMING



Sri Eshwar
College of Engineering
An Autonomous Institution
Affiliated to Anna University



Kondampatti (Post), Vadasithur (Via), Coimbatore - 641 202.

Department of

.....

Record work of Laboratory

Certified bonafide Record of work done by

Name : Roll No. :

Class : Branch :

Reg. No. :

Place : HOD Faculty In charge

Date :

University Register No.

Submitted for the University Practical Examination held on

.....

Internal Examiner

External Examiner

LIST OF EXPERIMENTS

EX. NO.	DATE	NAME OF THE EXPERIMENTS	PAGE NO.	MARKS (75)	SIGNATURE OF FACULTY
1.		Create a Python application that uses expressions and control flow statements to automate a common task. Ensure that your application is user-friendly and robust to different inputs. Suggested Problems: a) Swap two numbers without a temporary variable b) Quadratic Equation c) Valid Palindrome			
2.		Implement a Python program that simulates a real-world system or process using conditions and iterative loops. Suggested Problems: a) check whether an alphabet is a vowel or consonant b) sum of all even numbers from 0 to n, c) factorial of a number			
3.		Implementation of Strings in the program. Suggested Problems: a) Determine if string halves are alike, b) palindrome, c) character count d) replacing characters			
4.		Implementation of real-time/technical applications using Lists and Tuples. Suggested Problems: a) Minimum Index Sum of Two Lists, b) concatenate two lists index-wise, c) Tuple with the same product, d) Copy specific elements from one Tuple to a new tuple)			

5.		Implementation of real-time applications using Set and Dictionaries. Suggested Problems: a) Magic Dictionary b) Longest Word in Dictionary c) Set Mismatch and Smallest Number in Finite Set			
6.		Implementation of Functions in the program. Suggested Problems: a) Factorial b) largest number in a list c) area of shape			
7.		Implementation of file-handling operations. Suggested Problems: a) copy from one file to another b) wordcount c) longest word			
8.		Implementation of applications of standard libraries. Suggested Problems: Handle scalars to work on the NumPy array, Insert values at random positions in an array, Convert the index of a series into a column of a data frame, Combine many series to form a data frame, Get frequency counts of unique items of a series, Union of two arrays, Convert a NumPy array to a data frame of a given shape, Plotting datasets.			
9.		Implementation of OOP concepts in Python			
10.		Mini Project: Develop a web application for any real-world problem with Django			
CONTENT BEYOND SYLLABUS					
11.		SCIKIT-LEARN			

Marks in Words :
Average :

SIGNATURE OF THE FACULTY

Ex. No: 1a	SWAP WITHOUT A TEMPORARY VARIABLE
Date:	

AIM:

To write a python program to swap without a temporary variable.

ALGORITHM:

Step 1: Read the values of the two variables, a and b.

Step 2: Print the values of a and b before swapping.

Step 3: Perform the swap:

Step 4: Assign the sum of a and b to variable a.

Step 5: Assign the difference between the new value of a and b to variable b.

Step 6: Assign the difference between the new value of a and b to variable a again.

Step 7: Print the values of a and b after swapping.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 1b	QUADRATIC EQUATION
Date:	

AIM:

To write a python program to solve the Quadratic Equation.

ALGORITHM:

Step 1: Read the coefficients a, b, and c of the quadratic equation.

Step 2: Calculate the discriminant, given by the formula: $\text{discriminant} = b^2 - 4ac$.

Step 3: If the discriminant is greater than zero, calculate the two solutions:

- $\text{solution1} = (-b - \sqrt{\text{discriminant}}) / (2a)$
- $\text{solution2} = (-b + \sqrt{\text{discriminant}}) / (2a)$

Step 4: If the discriminant is equal to zero, calculate a single solution:

- $\text{solution} = -b / (2a)$

Step 5: If the discriminant is negative, print that the equation has no real solutions.

Step 6: Print the solutions if they exist.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 1c	VALID PALINDROME
Date:	

AIM:

To write a python program to solve the valid palindrome.

ALGORITHM:

Step 1: Read the input string.

Step 2: Initialize two pointers, start and end, pointing to the beginning and end of the string, respectively.

Step 3: Repeat the following until start is less than or equal to end:

- Skip non-alphanumeric characters at start and end.
- If the characters at indices start and end are not equal, the string is not a valid palindrome. Print the result and exit.
- Increment start by 1 and decrement end by 1.

Step 4: If the loop completes without any mismatched characters, the string is a valid palindrome.

Step 5: Print the result.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 2a	CHECK WHETHER AN ALPHABET IS A VOWEL OR CONSONANT
Date:	

AIM:

To write a python program to check whether an alphabet is a vowel or consonant..

ALGORITHM:

Step 1: Read the input alphabet.

Step 2: Convert the input alphabet to lowercase.

Step 3: Check if the input alphabet is equal to any of the vowel characters ('a', 'e', 'i', 'o', 'u')

- $\text{solution1} = (-b - \sqrt{\text{discriminant}}) / (2a)$

- $\text{solution2} = (-b + \sqrt{\text{discriminant}}) / (2a)$

Step 4: If the input alphabet is equal to any vowel character, it is a vowel. Print the result.

Step 5: If the input alphabet is not equal to any vowel character, it is a consonant. Print the result.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 2b	SUM OF ALL EVEN NUMBERS FROM 0 TO N
Date:	

AIM:

To write a python program to sum of all even numbers from 0 to n.

ALGORITHM:

Step 1: Read the input number n.

Step 2: Initialize a variable sum to store the sum of even numbers, starting from 0.

Step 3: Initialize a variable i to 0.

Step 4: Repeat the following until I is less than or equal to n:

- If I is an even number, add i to sum.
- Increment i by 2 to move to the next even number.

Step 5: Print the value of sum.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 2c	FACTORIAL OF A NUMBER
Date:	

AIM:

To write a python program to find the factorial of a number.

ALGORITHM:

Step 1: Read the input number n.

Step 2: Initialize a variable factorial to 1.

Step 3: Repeat the following until n is greater than 1:

- Multiply factorial by n.
- Decrement n by 1.

Step 4: Print the value of factorial.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 3a	DETERMINE IF STRING HALVES ARE ALIKE
Date:	

AIM:

To write a python program to implement string to determine if string halves are alike.

ALGORITHM:

Step 1: Get a string

Step 2: Divide a string in to two half

Step 3: Count the number of vowels in the two strings

Step 4: If count is equal then print True.

Step 5: If count is not equal then print False

PROGRAM:**OUTPUT:****RESULT:**

Ex. No: 3b	CONCATENATE TWO LISTS INDEX - WISE
Date:	

AIM:

To write a python program to solve the valid palindrome.

ALGORITHM:

Step 1: Read the input string.

Step 2: Initialize two pointers, start and end, pointing to the beginning and end of the string, respectively.

Step 3: Repeat the following until start is less than or equal to end:

- Skip non-alphanumeric characters at start and end.
- If the characters at indices start and end are not equal, the string is not a valid palindrome. Print the result and exit.
- Increment start by 1 and decrement end by 1.

Step 4: If the loop completes without any mismatched characters, the string is a valid palindrome.

Step 5: Print the result.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 3c	CHARACTER COUNT
Date:	

AIM:

To write a python program to count the characters.

ALGORITHM:

Step 1: Get a string

Step 2: Initialize count to zero

Step 3: Count each character/alphabet except space

Step 4: Increment count

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 3d	REPLACING CHARACTERS
Date:	

AIM:

To write a python program to replace character in given string.

ALGORITHM:

Step 1: Get a string

Step 2: Use replace function to replace with new string

`string.replace(oldvalue, newvalue, count)`

Step 3: Print the new string

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 4a	MINIMUM INDEX SUM OF TWO LISTS
Date:	

AIM:

To write a python program using lists to find the minimum Index Sum of Two Lists.

ALGORITHM:

Step1: Create two empty lists

Step2: Iterate through the indices i in the range of the length of list1.

- Iterate through the indices j in the range of the length of list2.
- Check if the element at list1[i] is equal to the element at list2[j].
- If they are equal, calculate the sum of i and j and append it to the list.
- Append the element at list1[i] to the list

Step 3: find the minimum value t in the e list.

Step4: create an empty list.

Step5: Iterate through the indices c in the range of the length of e.

- Check if the element at e[c] is equal to t.
- If it is equal, append the element at a[c] to the i list.

Step6: Print the i list.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 4b	CONCATENATE TWO LISTS INDEX-WISE
Date:	

AIM:

To write a python program using lists to concatenate two lists index-wise.

ALGORITHM:

Step 1: Create an empty lists i.

Step 2: Iterate through the indices i in the range of the length of a.

- Concatenate the element at index i from a with the element at the same index from b.
- Append the concatenated string to the a list Step

3: Print the i list.

PROGRAM:**OUTPUT:****RESULT:**

Ex. No: 4c	TUPLE WITH SAME PRODUCT
Date:	

AIM:

To write a python program using Tuples with the same product.

ALGORITHM:

Step 1: Create an empty lists tuples_list to store the tuples.

Step 2: Iterate through the range of numbers from 1 to n (inclusive) with the variable

- Iterate through the range of numbers from a to n (inclusive) with the variable
- Check if the product of a and b is equal to n.
- If the condition is true, create a tuple (a, b) and append it to the tuples_list.

Step 3: Return the tuples_list.

PROGRAM:**OUTPUT:****RESULT:**

Ex. No: 4d	COPY SPECIFIC ELEMENTS FROM ONE TUPLE TO A NEW TUPLE
Date:	

AIM:

To write a python program using Tuples to copy specific elements from one tuple to a new tuple.

ALGORITHM:

Step 1: Create an empty list new_tuple to store the selected elements.

Step 2: Iterate through each index i in the indices_to_copy list..

- Access the element at index i from the original_tuple using original_tuple[i].
- Append the element to the new_tuple list.

Step 3: Convert the new_tuple list into a tuple using the tuple() function.

Step 4: Print the new_tuple.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 5a	MAGIC DICTIONARY
Date:	

AIM:

To write a python program to implement Magic Dictionary.

ALGORITHM:

Step 1: Create a class MagicDictionary with an empty dictionary self.dictionary as an instance variable. Step 2: Implement the add_word method:

- Take a word as input.
- Get the length of the word.
- Check if the length exists as a key in the dictionary.
- If it doesn't exist, add a new key-value pair where the key is the length and the value is an empty list.
- Append the word to the list corresponding to the length in the dictionary.

Step 3: Implement the search method:

- Take a word as input.
- Get the length of the word.
- Check if the length exists as a key in the dictionary.
- If it doesn't exist, return False as there are no words of that length.
- Get the list of words of the same length from the dictionary.
- Iterate through each word in the list.
- initialize a variable diff_count to track the number of differing characters between the given word and the current word from the list.
- Iterate through each character index in the range of the length of the word.
- Compare the characters at the corresponding indices in both words.
- If the characters are different, increment diff_count by 1.
- If diff_count exceeds 1, break out of the loop to optimize the search.
- If diff_count is exactly 1, return True as a valid match is found
- If no valid match is found, return False.

Step 4: Create an instance of the MagicDictionary class called magic_dict. Step 5: Use the add_word method to add words to the magic dictionary.

Step 6: Use the search method to search for words in the magic dictionary and print the results

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 5b	LONGEST WORD IN DICTIONARY
Date:	

AIM:

To write a python program to find the longest word in a Dictionary.

ALGORITHM:

Step 1: Define a function longest_word that takes a dictionary word_dict as input. Step 2: Initialize a variable longest to an empty string.

Step 3: Iterate through the values of the dictionary using a loop.

- For each word in the dictionary values, check if its length is greater than the length of the current longest word.
- If the length of the word is greater, update the value of longest to the current word.

Step 4: After iterating through all the words, return the value of longest. Step 5: In the main program:

- Create an empty dictionary dict1.
- Prompt the user to enter values for the dictionary using a loop.
- Assign each input value to the corresponding key in dict1.
- Call the longest_word function with dict1 as an argument and assign the result to ylongest.
- Print the value of ylongest as the longest word.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 5c	SET MISMATCH
Date:	

AIM:

To write a python program to find the set mismatch.

ALGORITHM:

Step 1: Create a function findErrorNums that takes a list nums as input. Step 2: Get the length of the nums list and assign it to the variable n.

Step 3: Create an empty set num_set to store unique numbers.

Step 4: Initialize a variable duplicate to 0 to track the duplicate number.

Step 5: Calculate the total sum of all numbers in the nums list using the sum() function and assign it to the variable total_sum.

Step 6: Iterate through each number num in the nums list.

- Check if the current number num is already present in the num_set.
- If it is, assign the num value to the duplicate variable.
- Add the num to the num_set.

Step 7: Calculate the missing number using the formula: $((n * (n + 1)) // 2) - (total_sum - duplicate)$.

Step 8: Return a list containing the duplicate number and the missing number as [duplicate, missing].

PROGRAM:**OUTPUT:****RESULT:**

Ex. No: 5d	SMALLEST NUMBER IN FINITE SET
Date:	

AIM:

To write a python program to find the Smallest Number in finite set.

ALGORITHM:

Step 1: Create a set number_set with the given values.

Step 2: Find the smallest number in the set using the min() function and assign it to the variable smallest_number.

Step 3: Print the value of smallest_number along with an appropriate message.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 6a	FACTORIAL
Date:	

AIM:

To write a python program to implement the functions in the factorial program

ALGORITHM:

Step 1: Start with a positive integer n.

Step 2: If n is 0 or 1, return 1 (as the factorial of 0 and 1 is defined as 1).

Step 3: Otherwise, recursively calculate the factorial by multiplying n with the factorial of n - 1.

Step 4: Repeat step 3 until the base case is reached.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 6b	LARGEST NUMBER IN A LIST
Date:	

AIM:

To write a python program to implement the functions in the largest number in a list

ALGORITHM:

Step 1: Start with a list of numbers.

Step 2: Initialize a variable largest with the first element of the list.

Step 3: Iterate through each element in the list.

Step 4: If the current element is greater than largest, update largest to the current element.

Step 5: Repeat steps 3-4 for each element in the list.

Step 6: After iterating through all elements, largest will contain the largest number in the list.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 6c	AREA OF SHAPE
Date:	

AIM:

To write a python program to implement the functions in the largest number in a list

ALGORITHM:

Step 1: Start with the name of the shape and the required parameters for calculating its area.

Step 2: Convert the shape name to lowercase for case-insensitive comparison.

Step 3: If the shape is a circle, calculate the area using the formula: $\text{area} = \pi * \text{radius}^2$, where π is a constant and radius is the provided parameter.

Step 4: If the shape is a rectangle, calculate the area using the formula: $\text{area} = \text{length} * \text{width}$, where length and width are the provided parameters.

Step 5: If the shape is a triangle, calculate the area using the formula: $\text{area} = 0.5 * \text{base} * \text{height}$, where base and height are the provided parameters.

Step 6: If the shape is not recognized or supported, return None to indicate an invalid shape.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 7a	COPY FROM ONE FILE TO ANOTHER
Date:	

AIM:

To write a python program to copy from one file to another.

ALGORITHM:

Step1: The file test. txt is opened using the open() function.

Step2: Another file out.txt is opened using the open() function in the write mode using the f1stream.

Step 3: Each line in the file is iterated over using a for loop (in the input stream).

Step4: Each of the iterated lines is written into the output file.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 7b	WORD COUNT
Date:	

AIM:

To write a python program to count the word.

ALGORITHM:

Step1:Initialize the input string

Step 2: Print the original string.

Step3:Use the 'char.Split()'method to count the number of spaces in the string and add1 to it to get the count of words.

Step4: Print the count of words.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 7c	LONGEST WORD IN A FILE
Date:	

AIM:

To write a python program to find the longest word in a file.

ALGORITHM:

Step1:Create a variable to read the text file data using the read() function(reads the specified number of bytes from the file and returns them. ...

Step2:Find the length of the longest word using the len() (The number of items in an object is returned by the len() method.

PROGRAM:

OUTPUT:

RESULT:

Ex. No: 8	IMPLEMENTATION OF APPLICATIONS OF STANDARD LIBRARIES
Date:	

AIM:

To implement different applications using the libraries Numpy,Pandas and Matplotlib

ALGORITHM:

1. Import Numpy Package
2. Create two different numpy arrays with different dimensions
3. Perform arithmetic operations on arrays.
4. Insert element in random position in an array.
5. Perform union of two numpy arrays.
6. Import Pandas package
7. Convert the numpy array into data frame.
8. Create a Series, convert the series into data frame
9. Create a data frame multiple series.
10. Import the Matplotlib Package
11. Convert a plot from a dataset.

PROGRAM:

1. Handling Scalar Values in Numpy Arrays

OUTPUT:

2.Insert values at random positions in an array

OUTPUT:

3.Convert the index of a series into a column of a Data frame

OUTPUT:

4.Create DataFrame from multiple Series

OUTPUT:

5. GetFrequency counts of unique items of a series

OUTPUT:

6.Union of Two Numpy Arrays

OUTPUT:

7.Creation of Dataframe from a numpy array

OUTPUT:

8.Draw as catter plot from a dataset

OUTPUT:

RESULT:

Ex. No: 9	IMPLEMENTATION OF OOP CONCEPTS IN PYTHON
Date:	

AIM:

To write a python program for OOP concept.

ALGORITHM:

Step 1:import class and subclasses

Step 2: Define the __init__ method to initialize object attributes.

Step 3: Create a subclasses that inherit from a base class.

Step 4: Implement method overriding to allow subclasses to provide specific implementations of methods

Step 5: Use polymorphism to handle objects of different classes through a common interface.

Step 6: Ensure subclasses implement the abstract methods

Step 7:Use all the oops concepts and verify the output

PROGRAM:

Inheritance:

OUTPUT:

Polymorphism:

OUTPUT:

RESULT:

CONTENT BEYOND SYLLABUS	
Ex. No: 11	SCIKIT-LEARN
Date:	

AIM:

To perform program that performs classification using the Naive Bayes classifier from scikit-learn. This program uses the famous Iris dataset for training and evaluating the classifier:

PROGRAM:

OUTPUT :

RESULT:

